C200 Programming Assignment №8

Dr. M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

March 29, 2024

Please please start early on this HW. The HW is due on **Friday, April, 05 at 10:59 PM EST**. Please commit, push to Github and submit your work to the Autograder before the deadline.

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).

2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.

3. Make sure that the **code does not have infinite loop (that never exits) or an endless recursion (that never completes)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.

4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.

5. Once you are done testing your code, comment out the tests i.e. the code under the __name__ == "__main__" section.

Problem 1: sickit Library

In the previous homework we looked at linear regression coding the solution for a univariate problem. We found that the “best" values for our model were:

$$
\begin{aligned}
y(x) &= \hat{m}x + \hat{b} && (1)\\
&= 0.125x + 67.5 && (2)\\
R^2 &= 0.298 && (3)
\end{aligned}
$$

Scikit-learn is among the most popular free machine learning library for Python. Currently it is almost always required if someone is working in any data-intensive fields like computational science, computational business. In this problem we will use their package to (1) redo our homework problem (2) do a larger problem. Please visit: `https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html` to read about the linear regression tools available. Make sure you pay attention to the types it uses. We will return a structure that not only includes the slope, intercept, and R^2 (as we did in the previous homework), but also the model object for prediction. The following code:

```
data6 = get_data("", "payrollwins.txt")
print(f"Model built from parameters applied to 10: {(lambda x:0.1250*x + 67.498)(10)}")
plt.plot([x for x,_ in data6],[y for _,y in data6],'ro')
m_hat, b_hat, R_sq, model = my_scikit_LR(data6)
print(f"m_hat: {m_hat}, b_hat: {b_hat}, R^2: {R_sq}")
plt.plot([x for x,_ in data6],model(np.array([[x] for x,_ in data6])),'b')
# Following line make use of the prediction object to predit output for 10
print(f"Scikit Model applied to 10: {model(np.array([[10]]))[0][0]}")
plt.xlabel("$M cost")
plt.ylabel("Wins")
plt.title(f"Wins as function of $M cost R^2 = {round(R_sq,3)}")
plt.show()
```

has output:

```
 Model built from parameters applied to 10: 68.748
m_hat: 0.12501409085785145, b_hat: 67.4984922782099, R^2: 0.29851298993160025
Scikit Model applied to 10: 68.74863318678841
```

Observe that we can build the model too–this is nothing more than a λ expression. The plot is shown below.

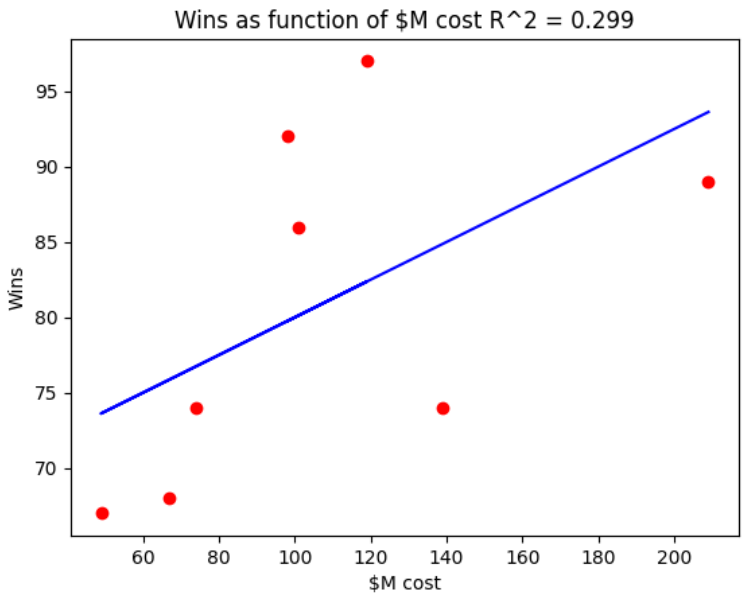

Figure 1: Using scikit-learn allows for much less code.

Programming Problem 1: scikit-learn Linear Regression

- The function takes data and returns slope,intercept,R^2, and the model. Use only this module **from sklearn.linear_model import LinearRegression**
- Instructions for installing sklearn are shared in the starter code.

Problem 2: Recursion and Tail Recursion

You worked with these last homework. For this homework, you'll write the tail recursion and while loop. This is a reminder that in the starter code, we are providing you with the regular recursion code, and the functions you must implement must utilize tail recursion and a while loop, In your starter code, function names will end in _t, _w, if the function is to be implemented using tail recursion, a while loop, respectively.

$$
\begin{aligned}
p(0) &= 10000 && (4)\\
p(n) &= p(n-1) + 0.02p(n-1) && (5)
\end{aligned}
$$

$$
\begin{aligned}
c(1) &= 9 && (6)\\
c(n) &= 9c(n-1) + 10^{n-1} - c(n-1) && (7)
\end{aligned}
$$

$$
\begin{aligned}
d(0) &= 1 && (8)\\
d(n) &= 3d(n-1) + 1 && (9)\\
& && (10)
\end{aligned}
$$

Programming Problem 2: Recursion, Tail Recursion, and While

- For reach function you'll write the tail recursive form and while.
- We have added the signature to help you–in particular, c(n) requires two accumulators.

Problem 3: Recursion

While equation 13 seems to be very complex, you have all the tools you need: functionality of sigma can be obtianed by using the range function, the c_2( ), function is provided in the starter code and it provide the functionality of choosing k values out of n. You are explicitly allowed (and encouraged) to use both sum( ) and list comprehension.

$$B_0 = 1 \tag{11}$$

$$B_n = \frac{-\sum_{k=0}^{n-1} \binom{n+1}{k} B_k}{n+1} \tag{12}$$

$$B(1) = \frac{-\sum_{k=0}^{0} \binom{2}{k} B_0}{1+1} \tag{13}$$

$$= -\frac{\binom{2}{0} 1}{2} \tag{14}$$

$$= -.5 \tag{15}$$

$$B(2) = -\frac{[\sum_{k=0}^{1} \binom{3}{k} B_k]}{3} \tag{16}$$

$$= -\frac{[\binom{3}{0} B_0 + \binom{3}{1} B_1]}{3} \tag{17}$$

$$= -\frac{[1(1) + 3(-.5)]}{3} \tag{18}$$

$$= -\frac{[1 - 1.5]}{3} = .5/3 = .1\bar{6}$$

$$B(3) = -\frac{[\sum_{k=0}^{2} \binom{4}{k} B_k]}{4} \tag{19}$$

$$= -\frac{[\binom{4}{0} B_0 + \binom{4}{1} B1 + \binom{4}{2} B2]}{4} \tag{20}$$

$$= -\frac{[1(1) + 4(-.5) + 6(.1\bar{6})]}{4} \tag{21}$$

$$= -\frac{[1 - 2 + 1]}{4} = 0 \tag{22}$$

Here are some outputs:

output

```
B(0) = 1
B(1) = -0.5
B(2) = 0.16666666666666666
B(3) = -0.0
B(4) = -0.033333333333333305
B(5) = -7.401486830834377e-17
```

Programming Problem 3: Recursion

- Complete the function B() using recursion.
- You can (and are encouraged to) use sum().
- *Hint*: List comprehension is useful here!

Problem 4: Approximating the Derivative

Calculus is used in virtually every field and is fundamental to understanding AI. In this problem we approximate the derivative which is instantaneous change. We'll focus on univariate functions initially:

$$f'(x) = \lim_{\epsilon \to 0} \frac{f(x+\epsilon) - f(x)}{\epsilon} \tag{23}$$

This is really a λ function, since we're given a function f and a value ϵ and returning a function f' that has x as an argument. We can approximate $f'(x)$ shown here:

$$f'(x) \approx \frac{f(x+\epsilon) - f(x-\epsilon)}{\epsilon} \tag{24}$$

We will write a function called derivative that takes a function and epsilon and returns a lambda function. for example,

$$f(x) = x^2 - 3x \tag{25}$$

$$f'(x) = 2x - 3 \tag{26}$$

$$f'(2) = 4 - 3 = 1 \tag{27}$$

```python
def derivative(f,epsilon):
    pass

def f(x):
    return x**2 - 3*x

data = 2
epsilon = 10e-8
print((derivative(f,epsilon)(data)))
f_prime = derivative((lambda x:x**2-3*x),epsilon)
print(f_prime(data))
```

has outputs:

```
0.9999999961429751
0.9999999961429751
```

The derivative is central to AI. Assume you have a real-valued function $f(x)$ and you want to find an optimal value; namely, you want to find for $x \in [a, b]$ the value that makes $f(x)$ the smallest. While we can't develope the full explanation here, we show that if you have an approximation x_i, then this rule will usually work:

$$x_{i+1} \leftarrow x_i - \eta f'(x_i) \tag{28}$$

The $0 < \eta < 1$ is the *learning rate*, and this is multiplied by the derivative at that point. Intuitively, you're moving down toward the smaller value, the derivative indicates by how much. Here's a simple example of using AI to find the mean. Assume we have a list of numbers $[x_1, x_2, \ldots, x_N]$. We know how to analytically find the mean, but we can use AI and the rule above:

$$x_{i+1} \leftarrow x_i - \eta f'(x_i) \tag{29}$$

$$f(x) = (\tfrac{1}{2}) \sum_{i=1}^{n} (x_i - x)^2 \tag{30}$$

The $(1/2)$ makes the problem easier and can be ignored. Using our computational approximation of derivative, we can write the rule as:

```
def update(w,data):
    eta = .2
    return w - eta*(derivative(lambda x:residuals(data,x),0.00001)(w))
```

We used λ to allow us to send the list of numbers as an additional argument. It can be ignored. Observe the code below as we use the rule.

```
N = 50
x = np.linspace(1,14,100)
gm = np.zeros(N)
r = np.zeros(N)
def mean(lst):
    s_ = 0
    N = len(lst)
    for i in range(N):
        s_ += lst[i]
    m_ = round(s_/N,2)
    return m_

 #this is eq. 30...it's also gives error in our approximation
def residuals(lst,m):
    s_ = 0
    N = len(lst)
    for i in range(N):
        s_ += (lst[i] - m)**2
    m_ = (1/2)*(s_/N)
```

```
        return m_
data = [1,1,2,6,10,12,13,14]

#update rule
def update(w,data):
    eta = .2
    epsilon = 0.00001
        return w - eta*(derivative(lambda x:residuals(data,x),epsilon)(w))

m_ = mean(data)

#successively applying the update rule
fmean = 1
for i in range(N):
    gm[i] = fmean
    r[i] = residuals(data,fmean)
    #print(fmean,residuals(data,fmean))
    fmean = update(fmean,data)

print(gm[-1])
print(m_)
plt.plot(gm,r,'bo')
for i in range(1,N):
    plt.plot([gm[i-1],gm[i]],[r[i-1],r[i]],'b--')
plt.plot(m_,residuals(data,m_),'ro')
plt.xlabel("Possible means")
plt.ylabel("Error")
plt.title(f"Using AI to search for the best mean {gm[-1]}")
plt.show()
```

produces Fig. 2, as successive approximations to the red value which is the mean.

Deliverables for Programming Problem 4

- Complete the derivate function

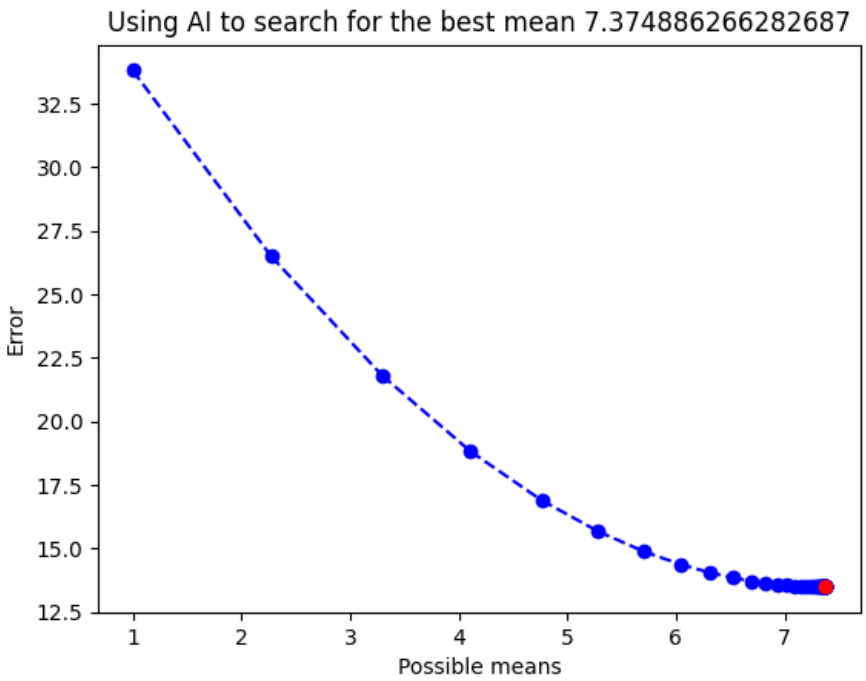

Figure 2: Using AI to find the mean

Problem 5: scikit-learn on bigger data

Now that you've used scikit-learn linear regression on small data, we'll use on larger data. Let's discuss R^2 some more. R^2 indicates how much variance the output is explained by the input. The rule of thumb is that $30\% \leq R^2$ indicates that the model is very poor. On the other hand $R^2 \geq 77\%$ means the model is good. The interval describes an unclear model. In the next homework we'll use a more recent extension of Linear Regression (LR) that takes into accout the distance x has from the model (the residual to remind you). The intuition is that the smaller residuals would inversely be weighted for x, since the farther away a value is, the less contribution it should have to building the model. When complete the plot you'll see is shown in Fig. 3. Note that R^2 is much better than the last model. You'll reusue your scikit function (from problem 1) exactly as is, but send it different data–to this end, observe that the code to get the data is different. The following code (which only differs with file):

```
path,name = "", "income_data.csv"
data4 = get_data_2(path,name)
plt.plot([x for x,_ in data4],[y for _,y in data4],'ro')
m_hat, b_hat, R_sq, model = my_scikit_LR(data4)
print(f"m_hat: {m_hat}, b_hat: {b_hat}, R^2: {R_sq}")
plt.plot([x for x,_ in data4],model(np.array([[x] for x,_ in data4↩
    ])),'b')
plt.xlabel("$M cost")
plt.ylabel("Wins")
plt.title(f"Wins as function of $M cost R^2 = {round(R_sq,3)}")
```

```
    plt.show()
```

generates the plot and outputs:

```
m_hat: 0.7138255122719709, b_hat: 0.20427039623160193, R^2: 0.7493217544889172
```

Figure 3: Plots of happiness as a function of income and the model (blue). There are 498 data points.

Hint: You can also revise the lab7 on file reading to understand how to read the files. Again, you won't be penalized if you use csv_reader or pandas (for file reading). It's upto you how you want to read-in the file.

Deliverables for Programming Problem 5

- Complete the get_data_2 changing the data.
- The data is from a file that contains scores for happiness as a function of income named `income_data.csv`
- Keep in mind that after you have tested your code, you must comment out the 'import matplotlib' and the plotting code given under the __main__ before submitting to the Autograder. Autograder can not draw graphical plots on the web browser so it will return an error if you do not comment out the plotting code/import matplotlib.

Problem 6: Candlesticks

Given a sequence of values $[v_0, v_1, \ldots, v_n]$ where $0, 1, \ldots, n$ represents time, a candlestick is the most popular visualization that is used to both summarize and inform financial analysts. The colors, red and green, indicate whether the valuation has negatively or positively changed, respectively. The line gives the range of valuations. The rectangle's height indicates the starting and ending value for that time period.

For a given sequence, a box and lines are drawn as shown in Fig. 4 (right)

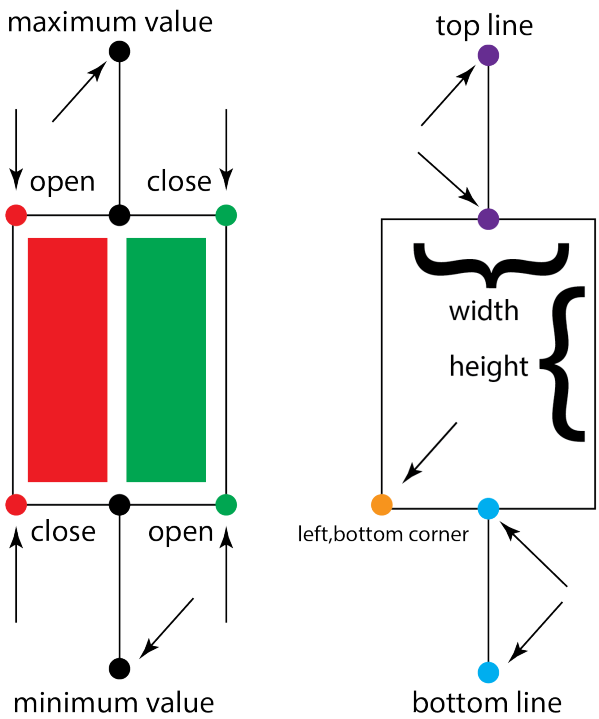

Figure 4: A candlestick image (right). The box is created by the opening and closing values. The top line is from the maximal value to the top middle of the box. The bottom line is from the minimal value to the bottom middle of the box. The color is determined by the open, close values. If open is greater than close, then the color is red; otherwise it's green. We use matplotlib patches. The candlestick image is determined by the rectangle and two lines. The rectangle is determined by the lower-left point (x, y), width, and height. The two lines are determined by two points computed from max, min, open, and close.

For any sequence we can quickly get the four values we need:

```
>>> import random as rn
>>> prices = [rn.randint(2,50) for _ in range(200)]
>>> open,close,max_p,min_p = prices[0],prices[len(prices)-1],max(↩
    prices),min(prices)
>>> open,close,max_p,min_p
(37, 2, 50, 2)
```

Using matplotlib's patches `https://matplotlib.org/stable/api/_as_gen/matplotlib.patches.`

`Rectangle.html` we can translate the four values into a candlestick, Fig. 4. (left). To plot a rectangle, we'll need the lower-lefthand corner (x,y), width, and height.

Note: The input is a collection of values contained inside a list so each sublist represents a candelstick. You will iterate over the contents of the list one-by-one and process each sublist insde the make() function to plot the candlesticks. The plotting is taken care of by the matplotlib patches but you still need to finish the make() function to return the values that are needed by the matplotlib patches. For quickly understanding how it works, try playing with the starter code of problem 5 under "__main__".

Hint: As each sublist is passed inside the make() function, you will first find the correct values of open, close, max_p and min_p (from the sublist), and then use those to values to calculate the values required to plot the candlestick. How to find, open, close, max_p and min_p is shown in the code listing above. The open and close will be used as the y values (i.e. the values on the y axis) and you have to find them from the sublist.

Let's look at the code:

```
data = [[20,15,32,10],[10,14,15,9],
        [22,23,27,9],[15,16,16,15],
        [26,12,30,2],[5,30,40,4]]

# open,close,max_p,min_p = 20, 15, 32, 10

# INPUTS ith candle, starting value of x, default width, and the four critical values: open, close, max\_p, min\_p.
# RETURN three tuples: (point, width, height, color), topline, bottomline
# point is the coordinates of the lower-left point x, y, width and height are numeric values and color will be a string of color ex. 'red' or 'green'
# topline ((xt0,yt0),(xt1,yt1)) line from max to top middle of box
# bottomline ((xb0,yb0),(xb1,yb1)) line from min to bottom middle of box

# When you see the code for testing Problem5 under __main__, you will see that the first three values of the first tuple i.e., point, width and height are passed as the first arguement of matplotlib.patches.Rectangle() function and the last value i.e. color is passed as the second arguement. Feel free to play around with the test code to get a feeling of how it is working. You will understand it much better with a bit of experimentation.

def make(i, start, width_default, d):
```

```
    pass

fig = plt.figure()
ax = fig.add_subplot(111)
start = 0
default_width = 10
for i in range(len(data)):

    candle_box, top_line, bottom_line = make(i,start,default_width, data[i])
    print(candle_box)
    ax.add_patch(matplotlib.patches.Rectangle(*candle_box[0:3],color = candle_box[3]))
    plt.plot([x for x,_ in top_line],[y for _,y in top_line],'black')
    plt.plot([x for x,_ in bottom_line],[y for _,y in bottom_line],'black')
    start += default_width

plt.xlabel("time (hour)")
plt.ylabel("Stock X price")
plt.title("Candlestick for Stock X mm/dd/yyyy")
plt.xlim([0, 60])
plt.ylim([0, 35])
plt.show()
```

will produce

```
((0, 15), 10, 5, 'red')
((10, 10), 10, 4, 'green')
((20, 22), 10, 1, 'green')
((30, 15), 10, 1, 'green')
((40, 12), 10, 14, 'red')
((50, 5), 10, 25, 'green')
```

and the plot in Figure 5. We can look at some of the Walmart data. Specifically,

```
    wms = pd.read_csv('walmart_stock.csv', index_col='Date',parse_dates=['Date'])
    # print(wms.head())
    wms_c = wms[['Open','Close','High','Low']]
    print(wms_c.iloc[0].values.flatten().tolist())
    N_stocks = 50
    for i in range(N_stocks):
        candle_box,top_line,bottom_line = make(i,start,default_width,wms_c.iloc[i].values.flatten().tolist())

```

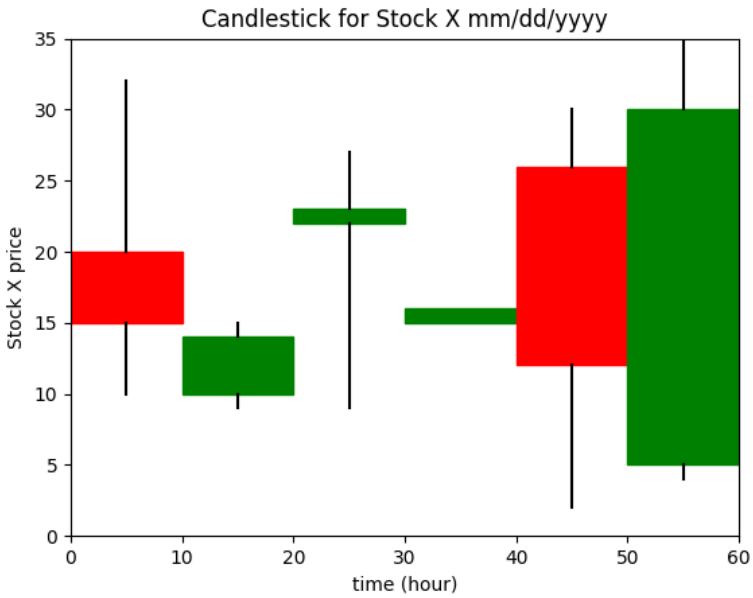

Figure 5: Six candlesticks

```
        # print(candle_box)
        ax.add_patch(matplotlib.patches.Rectangle(*candle_box[0:3],↩
            color = candle_box[3]))
        plt.plot([x for x,_ in top_line],[y for _,y in top_line],'↩
            black')
        plt.plot([x for x,_ in bottom_line],[y for _,y in bottom_line↩
            ],'black')
        start += default_width

    plt.xlabel("time day")
    plt.ylabel("Walmart price $")
    plt.title("Candlestick for Walmart")
    plt.xlim([0, N_stocks*default_width]) #depends on number
    plt.ylim([58, 63]) #depends on price
    plt.show()
```

that displays the first 50 days shown in Fig. 6.

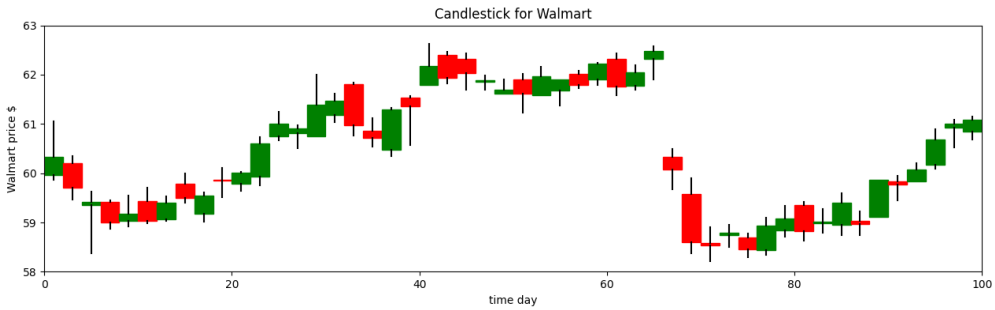

Figure 6: Candlestick for first 50 days

Deliverables for Programming Problem 6

- Complete the function make that returns three things: the data for the rectangle, the top line, and the bottom line.
- **Hint:** If you see the output produced by printing candel_box, notice how the value of start changes for each candle_box-this is what makes each candlebox shift in the plot so that each candelbox start at a specific distance after the previous one so that all of them looks seperated for easy visualization.
- Keep in mind that after you have tested your code, you must comment out the 'import matplotlib' and the plotting code given under the __main__ before submitting to the Autograder. Autograder can not draw graphical plots on the web browser so it will return an error if you do not comment out the plotting code/import matplotlib.

Problem 7: Permutations

In this problem, you'll take a list of numbers, then return the permutation. A permutation of a list π is a list $[\pi_0, \pi_1, \ldots, \pi_{\text{len}(\pi)!}]$ of all possible arrangements. To simplify this, we'll assume the list contains only unique positive integers of the form [1,2], [1,2,3], ... There are no contraints except you cannot use any modules that can produce permutations. The order of the permutation list is unimportant. For example, this code:

```
print(f"[2,3] creates {math.factorial(len([1,2]))} elements")
  print(permute([1,2]))

  print(f"[1,2,3] creates {math.factorial(len([1,2,3]))} elements")
  print(permute([1,2,3]))

  print(f"[1,2,3,4] creates {math.factorial(len([1,2,3,4]))} elements")
  print(permute([1,2,3,4]))
```

produces:

```
[2,3] creates 2 elements
[[1, 2], [2, 1]]
[1,2,3] creates 6 elements
[[1, 3, 2], [1, 2, 3], [2, 3, 1], [2, 1, 3], [3, 2, 1], [3, 1, 2]]
[1,2,3,4] creates 24 elements
[[1, 4, 3, 2], [1, 3, 4, 2], [1, 3, 2, 4], [1, 4, 2, 3],
[1, 2, 4, 3], [1, 2, 3, 4], [2, 4, 3, 1], [2, 3, 4, 1],
[2, 3, 1, 4], [2, 4, 1, 3], [2, 1, 4, 3], [2, 1, 3, 4],
[3, 4, 2, 1], [3, 2, 4, 1], [3, 2, 1, 4], [3, 4, 1, 2],
[3, 1, 4, 2], [3, 1, 2, 4], [4, 3, 2, 1], [4, 2, 3, 1],
[4, 2, 1, 3], [4, 3, 1, 2], [4, 1, 3, 2], [4, 1, 2, 3]]
```

We will leave it up to you how to implement–mine does not rely on recursion, but I implemented both. The autograder will use the non-recursive, since that is much faster, but you should do what works best. My approach uses the observation that a permutation is a subset of the

Cartesian product. For example, let $X = \{1, 2, 3\}$. Then $X \times X \times X = X^3$ is

$$
\begin{aligned}
X^3 &= \{(1,1,1),(1,1,2),(1,1,3),(1,2,1) && (31)\\
&\quad (1,2,2),(1,2,3),(1,3,1),(1,3,2) && (32)\\
&\quad (1,3,3),(2,1,1),(2,1,2),(2,1,3) && (33)\\
&\quad (2,2,1),(2,2,2),(2,2,3),(2,3,1) && (34)\\
&\quad (2,3,2),(2,3,3),(3,1,1),(3,1,2) && (35)\\
&\quad (3,1,3),(3,2,1),(3,2,2),(3,2,3) && (36)\\
&\quad (3,3,1),(3,3,2),(3,3,3)\} && (37)
\end{aligned}
$$

If you look above the output for [1,2,3], you'll see that it is a subset. The difference is we don't allow repeats. So, moving forward, if the count of any number in a tuple is greater than 1, then it can be discarded. List has a method .count(x) that returns the number of times x occurs in the list. You are free to use this. So the algorithm might be:

```
def permute(lst):
Input X = [x0, x1, ..., xn]
# [1,2], [1,2,3], [1,2,3,4],...
Output List of all permutations of X

def power(lst, m):
#returns power of a list
#Implement this
# power([1,2],2) → [[1,2],[1,1],[2,2],[2,1]]
def has_cnts(lst):
# returns True if any member occurs more than once
# Implement this–use .count() method
return list(filter(lambda x: not has_cnts(x),power(lst,len(lst))))
```

So you'll have to implement the power of a list (which does not exist in Python) and has_cnts which might take advantage of any([...]).

Deliverables for Programming Problem 7

- Complete the function **however** you'd like.

1 Programming pairs

mgrass@iu.edu, corrosar@iu.edu
jvgottum@iu.edu, mxrobins@iu.edu
pkurzawa@iu.edu, mitcchar@iu.edu
jhbruce@iu.edu, rimaeda@iu.edu
markdeng@iu.edu, haoztian@iu.edu

craftdc@iu.edu, bsai@iu.edu
sebcont@iu.edu, traepena@iu.edu
burkhel@iu.edu, jlopezmo@iu.edu
samjimen@iu.edu, archshar@iu.edu
sjbarry@iu.edu, colbstap@iu.edu
ryandaun@iu.edu, crschuit@iu.edu
cc96@iu.edu, breramer@iu.edu
jaxkatz@iu.edu, tiffyou@iu.edu
reychaud@iu.edu, arangi@iu.edu
chenkay@iu.edu, benserna@iu.edu
ecabebe@iu.edu, nmetlapa@iu.edu
renahan@iu.edu, auspiege@iu.edu
allennam@iu.edu, camlueck@iu.edu
askaul@iu.edu, ctsvetan@iu.edu
ejbusch@iu.edu, rsreedh@iu.edu
marbelli@iu.edu, brshelby@iu.edu
ykassahu@iu.edu, nicmill@iu.edu
abelani@iu.edu, ptapiaro@iu.edu
kespejo@iu.edu, tmukund@iu.edu
pafrost@iu.edu, vupatel@iu.edu
andhodge@iu.edu, aquintas@iu.edu
lrborows@iu.edu, njvinson@iu.edu
adevill@iu.edu, gravipat@iu.edu
cbremner@iu.edu, masoroki@iu.edu
chamblia@iu.edu, avaweish@iu.edu
cdinman@iu.edu, ishetty@iu.edu
shbivens@iu.edu, bpovinel@iu.edu
bcabral@iu.edu, jamread@iu.edu
njustin@iu.edu, jocpoiri@iu.edu
nabatja@iu.edu, nvoneper@iu.edu
belleli@iu.edu, trimedio@iu.edu
forlin@iu.edu, mgstack@iu.edu
aydclink@iu.edu, wagnebr@iu.edu
cjduffie@iu.edu, shmulla@iu.edu
blevidut@iu.edu, wardjohn@iu.edu
cfahy@iu.edu, surehma@iu.edu
nkushnir@iu.edu, mackeyr@iu.edu
wchorich@iu.edu, syeshala@iu.edu
jailjohn@iu.edu, amtsmith@iu.edu
langgrav@iu.edu, leoett@iu.edu
berrynm@iu.edu, ap59@iu.edu

conkeen@iu.edu, bfmark@iu.edu
jamlian@iu.edu, tryerrag@iu.edu
blberger@iu.edu, zstopher@iu.edu
zlangley@iu.edu, amatariy@iu.edu
ocrowder@iu.edu, nsmurthy@iu.edu
allentp@iu.edu, snlnu@iu.edu
balian@iu.edu, sparab@iu.edu
shaagore@iu.edu, alexpan@iu.edu
logellis@iu.edu, smunikun@iu.edu
reafrank@iu.edu, xsoto@iu.edu
auconner@iu.edu, jmatczak@iu.edu
cedonica@iu.edu, cmm23@iu.edu
zcarmich@iu.edu, camyee@iu.edu
daveanu@iu.edu, andmuchi@iu.edu
okorkmaz@iu.edu, javitto@iu.edu
sgudapur@iu.edu, olipfing@iu.edu
emiellis@iu.edu, ksreejit@iu.edu
beckab@iu.edu, mvsimi@iu.edu
nkhai@iu.edu, lomwean@iu.edu
ankotel@iu.edu, weinstab@iu.edu
anudivek@iu.edu, cmoverfe@iu.edu
tkonkim@iu.edu, msolomie@iu.edu
chodgman@iu.edu, nrimawi@iu.edu
cgreywal@iu.edu, smremmer@iu.edu
agunde@iu.edu, joprayit@iu.edu
adeglur@iu.edu, adpulipa@iu.edu
gmhowell@iu.edu, arstover@iu.edu
harrisdg@iu.edu, teswank@iu.edu
careyjo@iu.edu, peterant@iu.edu
mbadwe@iu.edu, johnsanc@iu.edu
madimond@iu.edu, ceprenti@iu.edu
achitred@iu.edu, jianlou@iu.edu
arkali@iu.edu, saravula@iu.edu
jkchaura@iu.edu, twirges@iu.edu
harmonnc@iu.edu, abbalokh@iu.edu
arkhal@iu.edu, peterabl@iu.edu
lasdodd@iu.edu, ashahi@iu.edu
ecanant@iu.edu, wangjinz@iu.edu
presharr@iu.edu, khwillar@iu.edu
hedavid@iu.edu, ymarrakc@iu.edu
rachguo@iu.edu, jusshah@iu.edu

ml82@iu.edu, iopfings@iu.edu
aadkhan@iu.edu, animerch@iu.edu
boding@iu.edu, ntorpoco@iu.edu
amgandhi@iu.edu, rahkpate@iu.edu
rcerauli@iu.edu, gstogdil@iu.edu
changali@iu.edu, etmillho@iu.edu
akumares@iu.edu, nsenthi@iu.edu
nbkotur@iu.edu, frvisco@iu.edu
jdemian@iu.edu, pratmodi@iu.edu
jakedeck@iu.edu, jomalek@iu.edu
sambereb@iu.edu, shubtiwa@iu.edu
kimyouh@iu.edu, katepate@iu.edu
zaycarey@iu.edu, ggriorda@iu.edu
camehugh@iu.edu, lawlin@iu.edu
tkalale@iu.edu, reddysk@iu.edu
nsfried@iu.edu, joebrahm@iu.edu
sdharmmi@iu.edu, kammich@iu.edu
anchinna@iu.edu, bwmoore@iu.edu
lineri@iu.edu, menonvi@iu.edu